

OAuth 2.0

Authorization Framework — Study Notes

4 Actors · Authorization Code Flow · All Grant Types
Access & Refresh Tokens · CSRF Protection · Security Rules

TOPICS COVERED

1. What is OAuth 2.0? (The Problem it Solves)

2. Four Actors: Owner, Client, Auth, Resource

3. Client Registration & Redirect URIs

4. Authorization Code Flow (Step by Step)

5. Token Exchange & Token Refresh

6. All Four Grant Types Compared

7. Access Token vs Refresh Token

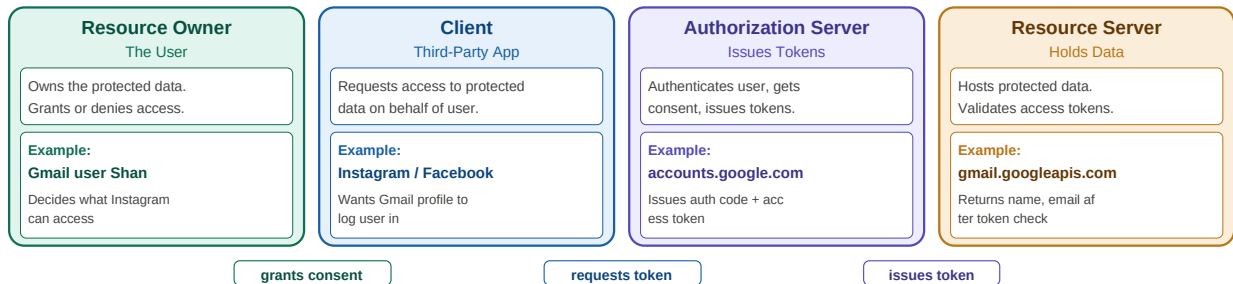
8. CSRF Protection via State Parameter

1. What is OAuth 2.0? The Problem it Solves	3
2. Four Actors in OAuth 2.0	3
3. Client Registration — Client ID, Secret, Redirect URI	4
4. Authorization Code Grant — Full Step-by-Step Flow	4
5. Why Two Steps? Auth Code + Token Exchange	5
6. All Four Grant Types Compared	5
7. Access Token vs Refresh Token vs Auth Code	6
8. State Parameter — CSRF Protection	6
9. Quick Revision & Interview Questions	7

1 & 2. What is OAuth 2.0? — Four Actors

OAuth 2.0 (Open Authorization) is an **authorization framework** — not an authentication protocol. It lets users grant third-party apps limited access to their data *without sharing their password*. The classic example: logging into Instagram using your Gmail account.

OAuth 2.0 — Four Actors How They Interact



OAuth 2.0 — What Problem Does It Solve?

Without OAuth:

- Instagram asks for your Gmail password
- You give Instagram your Gmail credentials
- Instagram now has FULL access to your Gmail
- If Instagram is hacked - your Gmail is compromised

With OAuth 2.0:

- + Instagram never sees your Gmail password
- + You grant only specific permissions (scope)
- + Gmail issues a limited-time access token
- + You can revoke access any time from Gmail settings

4 actors: Resource Owner (user) + Client (Instagram) + Auth Server (accounts.google.com) + Resource Server (Gmail API)

OAuth 2.0 is authorization (what can you access?), NOT authentication (who are you?). OpenID Connect (OIDC) adds authentication on top of OAuth 2.0.

The user never gives their Gmail password to Instagram. Gmail issues a limited, scoped, time-bound token instead.

Scopes define what the token can access: email, profile, contacts, calendar. Instagram only gets what it asks for and what the user approves.

Tokens can be revoked at any time from Google Account settings without changing your Gmail password.

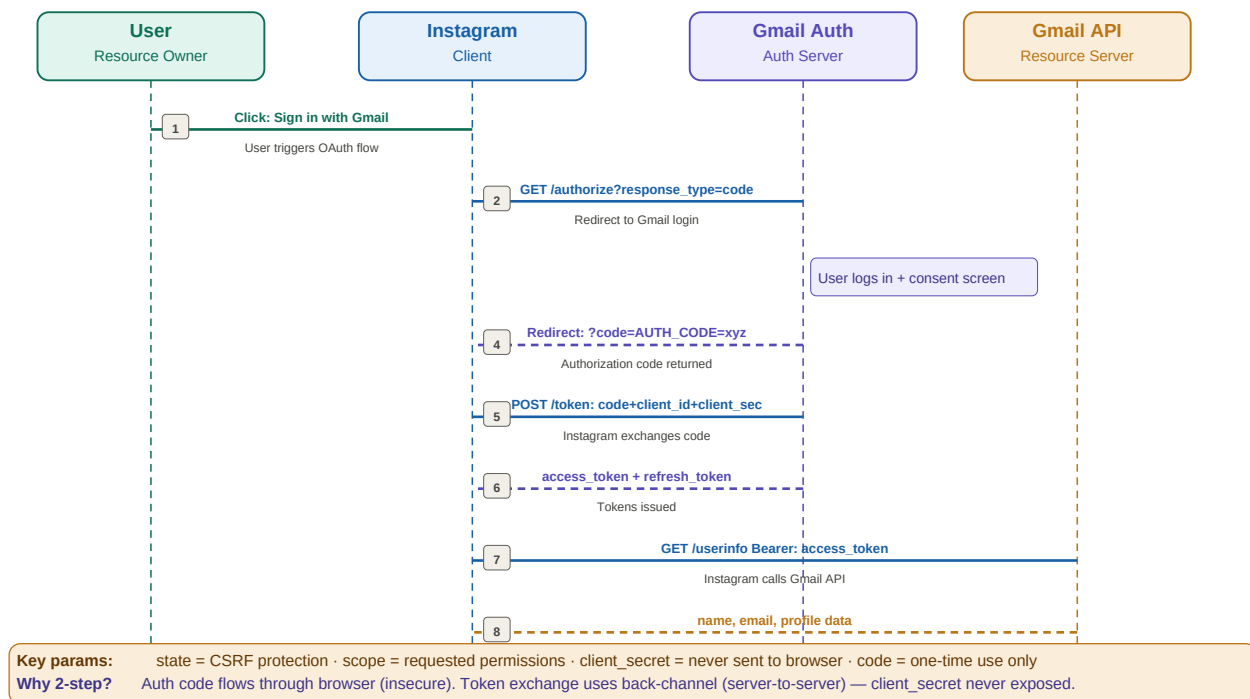
3 & 4. Client Registration & Authorization Code Flow

Step 0 — Client Registration (Done once before any user flow)

Before any user can log in via Gmail, Instagram must **register with Google's Authorization Server**. This happens once in the developer console.

What Instagram provides	What Google returns	Purpose
App name, website URL	Client ID (public)	Identifies the app on every request
Redirect URIs (up to 3)	Client Secret (private)	Proves app identity during token exchange
Requested scopes	Registration confirmation	Defines allowed permissions

Authorization Code Grant — Step-by-Step Flow



8-step Authorization Code flow: User->Instagram->Gmail Auth->consent->code->token exchange->API call->user data

5 & 6. Why Two Steps + All Four Grant Types

Why Authorization Code Grant Uses Two Steps

Step 1 — Auth code via browser redirect: The code travels through the browser (insecure channel). But it's useless alone — it cannot access anything by itself. Short-lived (~10 min), one-time use.

Step 2 — Token via back-channel (server-to-server POST): Instagram's backend directly calls Google's /token endpoint with the code + client_secret. This never touches the browser. The client_secret authenticates the request.

Result: The access token and refresh token are NEVER visible in the browser URL or logs. Only Instagram's server sees them. This is why Authorization Code Grant is the most secure flow.

OAuth 2.0 Grant Types — All Four Compared

Authorization Code	Implicit Grant	Password Credentials	Client Credentials
RECOMMENDED	LEGACY / AVOID	HIGH TRUST ONLY	SERVER-TO-SERVER
1. Client redirects user to	1. Client redirects to Auth	1. User gives username+password	1. No user involved at all
2. User logs in and consents	2. User logs in and consents	2. Client sends creds to A	2. Client sends client_id+
3. Auth Server returns auth code	3. Token returned directly	3. Auth Server validates c	3. Auth Server validates c
4. Client exchanges code for tokens	(no code exchange step)	4. Issues tokens directly	4. Issues token for the client
Issues: access_token + refresh_token	Issues: access_token only	Issues: access_token + refresh_token	Issues: access_token only
Refresh token: YES	Refresh token: NO	Refresh token: YES	Refresh token: NO
Use: Web apps, mobile apps	Use: Legacy SPAs (avoid)	Use: Trusted first-party apps	Use: Microservices, APIs
Most secure — use always	Token in URL = insecure. Deprecated.	Client sees password = high risk	Machine-to-machine, no user consent

Always use: Authorization Code Grant for all web and mobile apps.

Never use: Implicit Grant (deprecated, token exposed in URL). Avoid Password Grant unless first-party trusted app.

Auth Code: most secure, use always · Implicit: deprecated · Password: trusted apps only · Client Credentials: machine-to-machine

Grant Type	User Involved?	Auth Code?	Back-channel?	Refresh Token?	Use When
Authorization Code	YES — login + consent	YES	YES — most secure	YES	Web/mobile apps
Implicit	YES — login + consent	NO — token direct	NO — browser only	NO	Legacy SPAs only
Password Credentials	YES — gives password to client	NO	YES	YES	Trusted first-party
Client Credentials	NO — machine only	NO	YES	NO	Microservice APIs

7 & 8. Tokens, CSRF Protection & Security Rules

Tokens, State Parameter CSRF Protection

Access Token vs Refresh Token

Access Token
Short-lived (e.g. 1 hour)
Sent with every API request
If stolen: limited window of exposure

Refresh Token
Long-lived (days / permanent)
Used ONLY to get new access token
Never sent to Resource Server -- only Auth Server

Auth Code
One-time use, expires in ~10 mins
Exchanged for access+refresh tokens
Useless after first use -- replay attacks blocked

Token Refresh Flow:
1. Access token expires (401 Unauthorized)
2. Client sends refresh token to Auth Server
3. New access + refresh tokens issued

State Parameter -- CSRF Protection

Without state -- CSRF Attack:
1. Attacker gets their own auth code
2. Embeds code in link to victim's browser
3. Victim's session is tied to attacker's code
4. Victim unknowingly logs into attacker's account

With state -- Protected:
1. Client generates random state=abc123
2. Includes state in /authorize request
3. Auth server echoes state back in redirect
4. Client verifies: returned state == abc123?
5. Mismatch? Reject. Match? Proceed safely.

Key Security Rules

+ ALWAYS use state parameter
Prevents CSRF -- validate on every response

+ Keep client_secret server-side
Never expose in browser JS or mobile app

+ Use HTTPS everywhere
Tokens in transit must be encrypted

+ Short-lived access tokens
Limits damage if token stolen

+ Minimal scope
Request only needed permissions

+ Rotate refresh tokens
Issue new refresh token each use

3 token types + token refresh flow · CSRF attack without state vs safe flow with state · 6 security rules

Token	Lifetime	Where used	Sent to	If compromised
Auth Code	~10 minutes — one time	Exchanged for tokens	Auth Server only	Useless — already used or expired
Access Token	Short (1 hour typical)	Every API request	Resource Server	Attacker can access data for ~1 hour
Refresh Token	Long (days/permanent)	Token renewal only	Auth Server only	Attacker can get new access tokens

9. Quick Revision & Interview Questions

Quick Revision

- > OAuth 2.0 = authorization framework. Delegates access without sharing passwords.
- > OAuth 2.0 is NOT authentication. OpenID Connect (OIDC) adds identity on top of OAuth.
- > 4 actors: Resource Owner (user), Client (app), Authorization Server, Resource Server.
- > Client Registration: client gets Client ID (public) + Client Secret (private) + registers redirect URIs.
- > Authorization Code Grant: 8 steps. Auth code via browser, token via back-channel. Most secure.
- > Why two steps? Auth code is safe in browser (useless alone). Token exchange uses client_secret never visible in browser.
- > Scope: defines what permissions the client is requesting (email, profile, contacts).
- > State parameter: random string included in request and response. Validates response belongs to this request.
- > CSRF attack: attacker injects their auth code into victim's flow. State prevents this.
- > Access token: short-lived (1hr). Sent in every API request. Bearer token in Authorization header.
- > Refresh token: long-lived. Used only to get new access tokens. Never sent to Resource Server.
- > Auth code: one-time use, ~10 min. Worthless after exchange.
- > Implicit Grant: token directly in URL. Deprecated — never use. No refresh token.
- > Password Grant: client collects user password. Only for first-party trusted apps.
- > Client Credentials Grant: no user. Machine-to-machine. Microservices calling each other.
- > Keep client_secret on server only. Never in browser JS or mobile app binary.
- > Always use HTTPS. Token in plaintext over HTTP = instant compromise.

Interview Questions

- > 1. What is OAuth 2.0? What problem does it solve?
- > 2. What is the difference between authentication and authorization?
- > 3. Name the four actors in OAuth 2.0 and explain each with the Instagram/Gmail example.
- > 4. What is client registration? What is a Client ID vs Client Secret?
- > 5. Walk me through the Authorization Code Grant step by step.
- > 6. Why does Authorization Code Grant use two steps instead of returning the token directly?
- > 7. What is the state parameter? How does it prevent CSRF attacks?
- > 8. What is the difference between an access token and a refresh token?
- > 9. When would you use Client Credentials Grant instead of Authorization Code Grant?
- > 10. Why is the Implicit Grant deprecated?
- > 11. What is a scope in OAuth 2.0? Give three examples.
- > 12. What happens when an access token expires? Walk through the refresh flow.
- > 13. How does the Resource Server know the access token is valid?
- > 14. Design the OAuth flow for: a mobile app that wants to post tweets using Twitter API.

Key Terms

OAuth 2.0	Authorization framework — delegates access without sharing credentials
------------------	--

Resource Owner	User who owns the data and grants or denies access
Client	Third-party app requesting access (Instagram, Facebook)
Authorization Server	Issues tokens after authenticating user and getting consent
Resource Server	Hosts protected data, validates access tokens on each request
Client ID	Public identifier for the app. Sent in every authorization request.
Client Secret	Private secret. Used ONLY in back-channel token exchange. Never in browser.
Authorization Code	Short-lived one-time code. Exchanged for access+refresh tokens.
Access Token	Short-lived Bearer token. Sent with every API request.
Refresh Token	Long-lived token. Used only to get new access tokens.
Scope	Permissions the client requests. e.g. email, profile, contacts.
Redirect URI	URL the auth server sends code/token back to. Must be pre-registered.
State	Random value included in request and response. Prevents CSRF attacks.
CSRF	Cross-Site Request Forgery. State parameter prevents this in OAuth.
OIDC	OpenID Connect — identity layer on top of OAuth 2.0. Adds ID token.